

媒体与认知课程上机作业中期报告

陈天乐 无 25

2025 年 5 月 16 日

目录

1	项目背景	2
2	项目目标	2
3	实验任务	2
3.1	Task1: 图像编码器	2
3.2	Task2: 文本编码器	3
3.3	Task3: 对比损失	6
3.4	训练模型	7
3.5	Task4: 结果分析与可视化	10
4	实验总结	13
5	源代码	14

1 项目背景

多模态学习（Multimodal Learning）是当前人工智能领域的研究热点，其中视觉与语言的融合任务，如图文匹配、图文检索、图文生成等，具有广泛的应用前景。

CLIP（Contrastive Language-Image Pretraining）是 OpenAI 提出的多模态预训练方法，利用图像-文本对比学习，使模型能自动将图像和文本嵌入到同一语义空间中，进而支持多种下游任务。

本项目旨在指导学生构建一个简化版本的 CLIP 模型，掌握对比学习的基本方法和图文模态对齐的核心原理。

2 项目目标

1. 搭建一个具备图文对比学习能力的基础模型；
2. 使用 PyTorch 实现图像编码器与文本编码器的双塔结构；
3. 使用 InfoNCE 损失进行图文嵌入对齐训练；
4. 分析模型在多模态语义对齐方面的效果及可视化结果。

3 实验任务

本次实验使用的是 Flickr8k 数据集，数据格式为图像 + 文字描述（1-5 条）。模型结构如下

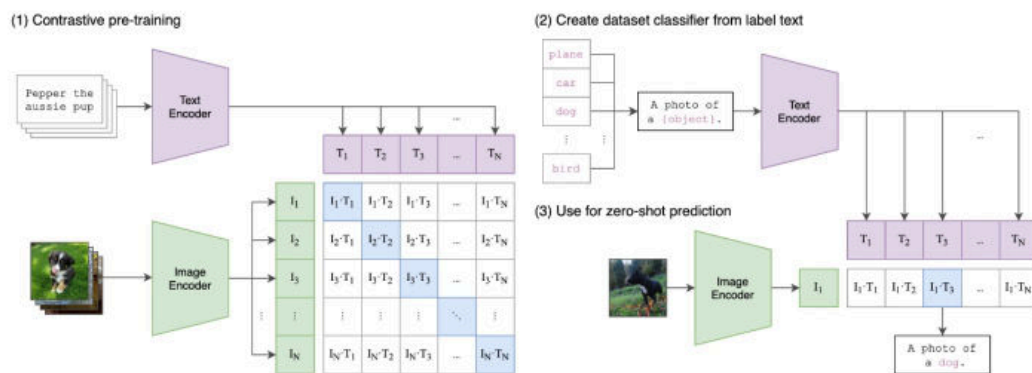


图 1: 模型结构

从示意图可以看出，该模型分为文本编码处理和图像编码处理两大部分。通过 ResNet 对图像进行特征提取，同时用 LSTM 或者 Transformer 模型（可在 text_encoder.py 中调整，本次实验使用 Transformer 模型）对输入文本提取文本表示，并形成图文嵌入矩阵。接着使用 InfoNCE 损失函数对图文嵌入对进行正负对比学习，

3.1 Task1: 图像编码器

该任务是将 ResNet 模型中的 forward 函数完善，代码如下：

```

1 # ----- 任务一：实现 BasicBlock 模块 -----
2
3 class BasicBlock(nn.Module):
4     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
5         super().__init__()
6         # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
7         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
8                                 padding=1, bias=False)
9
10        # 【填空区域】：对 conv1 的输出接 BatchNorm
11        self.bn1 = nn.BatchNorm2d(out_channels)
12
13        # 定义 ReLU 激活, inplace 设为 True
14        self.relu = nn.ReLU(inplace=True)
15
16        # 【填空区域】：实现第二个 3x3 卷积层, 步长默认为1
17        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
18                                bias=False)
19
20        # conv2 的输出接 BatchNorm
21        self.bn2 = nn.BatchNorm2d(out_channels)
22
23        self.downsample = downsample
24        # 请完成forward函数
25        def forward(self, x):
26
27            identity = x
28
29            out = self.conv1(x)
30            out = self.bn1(out)
31            out = self.relu(out)
32
33            out = self.conv2(out)
34            out = self.bn2(out)
35
36            # 如果需要下采样, 对 identity 也进行转换
37            if self.downsample is not None:
38                identity = self.downsample(x)
39
40            out = out + identity # 残差连接
41            out = self.relu(out)
42
43            return out

```

3.2 Task2: 文本编码器

该任务是将 LSTM 和 Transformer 模型中的代码完善，代码如下：

```

1 # ----- 任务一：实现 BasicBlock 模块 -----
2
3 class BasicBlock(nn.Module):
4     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
5         super().__init__()
6         # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
7         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
8                                 padding=1, bias=False)
9
10        # 【填空区域】：对 conv1 的输出接 BatchNorm
11        self.bn1 = nn.BatchNorm2d(out_channels)

```

```

12     # 定义 ReLU 激活, inplace 设为 True
13     self.relu = nn.ReLU(inplace=True)
14
15     # 【填空区域】: 实现第二个 3x3 卷积层, 步长默认为1
16     self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
17                             bias=False)
18
19     # conv2 的输出接 BatchNorm
20     self.bn2 = nn.BatchNorm2d(out_channels)
21
22     self.downsample = downsample
23     # 请完成 forward 函数
24     def forward(self, x):
25
26         identity = x
27
28         out = self.conv1(x)
29         out = self.bn1(out)
30         out = self.relu(out)
31
32         out = self.conv2(out)
33         out = self.bn2(out)
34
35         # 如果需要下采样, 对 identity 也进行转换
36         if self.downsample is not None:
37             identity = self.downsample(x)
38
39         out = out + identity # 残差连接
40         out = self.relu(out)
41
42         return out

```

```

1 class LSTMTextEncoder(nn.Module):
2     def __init__(self, vocab_size, embed_dim=256, hidden_dim=256, num_layers=1, padding_idx=0)
3         :
4         super().__init__()
5
6         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
7         self.hidden_dim = hidden_dim
8
9         self.W_i = nn.Linear(embed_dim, hidden_dim)
10        # 初始化输入门参数, U_i: 上一个隐状态至隐状态
11        self.U_i = nn.Linear(hidden_dim, hidden_dim)
12
13        # 初始化遗忘门参数, W_f: 输入至隐状态
14        self.W_f = nn.Linear(embed_dim, hidden_dim)
15        # 初始化遗忘门参数, U_f: 上一个隐状态至隐状态
16        self.U_f = nn.Linear(hidden_dim, hidden_dim)
17
18        # 初始化输出门参数, W_o: 输入至隐状态
19        self.W_o = nn.Linear(embed_dim, hidden_dim)
20        # 初始化输出门参数, U_o: 上一个隐状态至隐状态
21        self.U_o = nn.Linear(hidden_dim, hidden_dim)
22
23        # 初始化候选状态参数, W_c: 输入至隐状态
24        self.W_c = nn.Linear(embed_dim, hidden_dim)
25        # 初始化候选状态参数, U_c: 上一个隐状态至隐状态
26        self.U_c = nn.Linear(hidden_dim, hidden_dim)
27
28        # 全连接层将最终的隐藏状态映射到 embed_dim (目标嵌入空间)
29        self.fc = nn.Linear(hidden_dim, embed_dim)

```

```

30 #请完成forward函数
31 def forward(self, captions):
32     """
33     captions: [B, T], 表示批次 B 中每个句子的 token id 序列, T 为序列长度。
34     """
35     word_embedding = self.embedding(captions) # [B, T, E]
36     B, T, E = word_embedding.shape
37
38     h = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
39     c = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
40
41     for t in range(T):
42         x_t = word_embedding[:, t, :]
43
44         i_t = torch.sigmoid(self.W_i(x_t) + self.U_i(h)) # 输入门
45         f_t = torch.sigmoid(self.W_f(x_t) + self.U_f(h)) # 遗忘门
46         o_t = torch.sigmoid(self.W_o(x_t) + self.U_o(h)) # 输出门
47         c_hat_t = torch.tanh(self.W_c(x_t) + self.U_c(h))
48
49         c = f_t * c + i_t * c_hat_t # 更新细胞状态
50         h = o_t * torch.tanh(c) # 更新隐藏状态
51
52
53     # 将最终隐状态 h 通过全连接层映射至目标嵌入空间
54     out = self.fc(h) # [B, embed_dim]
55     # 对输出结果进行 L2 正则化归一化, 确保嵌入向量单位化
56     return F.normalize(out, p=2, dim=1)

```

```

1
2 # ----- 填空题 1: 实现 MultiHeadSelfAttention 模块 -----
3 class MultiHeadSelfAttention(nn.Module):
4     def __init__(self, embed_dim, num_heads):
5         super().__init__()
6         assert embed_dim % num_heads == 0, "embed_dim 必须能整除 num_heads"
7         self.num_heads = num_heads
8         self.head_dim = embed_dim // num_heads
9         self.qkv_proj = nn.Linear(embed_dim, embed_dim * 3)
10        self.out_proj = nn.Linear(embed_dim, embed_dim)
11
12    #请完成forward函数
13    def forward(self, x, mask=None):
14        # x: [B, T, D], D=embed_dim
15        B, T, D = x.size()
16        qkv = self.qkv_proj(x) # [B, T, 3D]
17        q, k, v = torch.chunk(qkv, 3, dim=-1)
18
19        # 拆分
20        q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2) # [B, nh, T, hd]
21        k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
22        v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
23
24        #计算注意力分数
25        attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim) # [B, nh, T, T]
26
27        if mask is not None:
28            attn = attn.masked_fill(mask.unsqueeze(1).unsqueeze(1) == 0, float('-inf'))
29            attn = F.softmax(attn, dim=-1)
30
31        # 计算注意力输出
32        attn_output = attn @ v # [B, nh, T, hd]
33        attn_output = attn_output.transpose(1, 2).contiguous().view(B, T, D) # [B, T, D]
34
35        #通过输出投影层输出结果

```

```

35         return self.out_proj(attn_output)
36
37 class TransformerBlock(nn.Module):
38     def __init__(self, embed_dim, num_heads):
39         super().__init__()
40         self.attn = MultiHeadSelfAttention(embed_dim, num_heads)
41         self.ln1 = nn.LayerNorm(embed_dim)
42         # 初始化前馈网络, 先 Linear(embed_dim, embed_dim*4) -> ReLU -> Linear(embed_dim*4,
43         embed_dim)
44         self.ff = nn.Sequential(
45             nn.Linear(embed_dim, embed_dim * 4),
46             nn.ReLU(),
47             nn.Linear(embed_dim * 4, embed_dim)
48         )
49         self.ln2 = nn.LayerNorm(embed_dim)
50
51     def forward(self, x, mask=None):
52         # 残差连接: 先 LayerNorm, 再 MultiHeadSelfAttention, 再加上输入
53         # 计算自注意力块输出, 并加上原输入, 实现残差连接
54         x = x + self.attn(self.ln1(x), mask)
55         # 同理, 对于前馈网络: 先 LayerNorm, 再前馈计算, 再加上输入
56         # 计算前馈网络输出, 并加上残差
57         x = x + self.ff(self.ln2(x))
58         return x
59
60 # ----- 填空题 2: 实现 TransformerTextEncoder 模块 -----
61 class TransformerTextEncoder(nn.Module):
62     def __init__(self, vocab_size, embed_dim=256, num_heads=4, num_layers=4, padding_idx=0,
63         max_len=100):
64         super().__init__()
65         # 初始化嵌入层
66         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
67         # 初始化位置编码器, 参数为 embed_dim 和 max_len
68         self.pos_encoder = PositionalEncoding(embed_dim, max_len=max_len)
69         self.layers = nn.ModuleList([TransformerBlock(embed_dim, num_heads) for _ in range(
70             num_layers)])
71         self.fc = nn.Linear(embed_dim, embed_dim)
72
73     # 请完成 forward 函数
74     def forward(self, captions):
75         # captions: [B, T]
76
77         # 输入嵌入和位置编码
78         x = self.embedding(captions) # [B, T, D]
79         x = self.pos_encoder(x) # [B, T, D]
80
81         # 通过多层 Transformer 块
82         for layer in self.layers:
83             x = layer(x) # [B, T, D]
84
85         # 全局平均池化
86         x = x.mean(dim=1) # [B, D]
87         out = self.fc(x)
88         return F.normalize(out, p=2, dim=1)

```

3.3 Task3: 对比损失

该任务是完善 InfoNCE 损失函数, 目标是: 同一图文对相似度最大, 且不同对之间的相似度最小。代码如下:

```

1 # 请在这里写出对比损失函数
2 def contrastive_loss(image_embeds, text_embeds, temperature=0.07):

```

```

3     """
4     image_embeds, text_embeds: [batch, embed_dim]
5     """
6     device = image_embeds.device
7     batch_size = image_embeds.shape[0]
8
9     logits = torch.matmul(image_embeds, text_embeds.T) / temperature
10
11     # 创建正样本标签 [0,1,2,...,batch-1]
12     labels = torch.arange(batch_size, device=image_embeds.device)
13
14     # 计算图像->文本和文本->图像两个方向的交叉熵损失
15     loss_i2t = F.cross_entropy(logits, labels)
16     loss_t2i = F.cross_entropy(logits.T, labels) # 转置矩阵计算反向
17
18     # 最终损失取平均
19     loss = (loss_i2t + loss_t2i) / 2
20
21     return loss

```

同时，为了计算验证集中的 loss，同样编写了 loss 函数：

```

1 def calc_val_loss(img_encoder, txt_encoder, dataloader, device):
2     img_encoder.eval() # 设置为评估模式
3     txt_encoder.eval() # 设置为评估模式
4     total_val_loss = 0 # 使用 total_val_loss 累加
5     with torch.no_grad(): # 在评估阶段不计算梯度
6         pbar = tqdm(dataloader, desc="计算验证损失", unit="batch", leave=False)
7         for images, captions_ids in pbar:
8             images = images.to(device)
9             captions_ids = captions_ids.to(device)
10
11             image_embeds = img_encoder(images)
12             text_embeds = txt_encoder(captions_ids)
13             loss = contrastive_loss(image_embeds, text_embeds) # 调用对比损失函数
14             total_val_loss += loss.item()
15             pbar.set_postfix(loss=loss.item()) # 显示当前batch的损失
16     avg_val_loss = total_val_loss / len(dataloader)
17     return avg_val_loss

```

3.4 训练模型

训练流程大致为：

1. 将图像与文本数据输入模型；
2. 利用图像和文本特征提取编码器提取特征信息；
3. 归一化后计算余弦相似度矩阵；
4. 计算损失并反向传播；
5. 每轮评估 Top-1 Accuracy / Recall@K 检索准确率，并将训练中得到的最优模型保存至本地（后缀为.pth 文件）。

在 train.py 中，为了保存一个 loss 的训练曲线，需要对每个训练周期得到的 train 和 val 的 loss 保存下来，保存在数组中后，待最后一个 epoch 训练完后绘制出 loss 曲线。在每一次训练结束时输出当前训练周期的 loss、recall 等参数；每一轮训练时会检测是否已经有模型数据，这样可以从上一次的训练中断处继续训练；设置一个 improve_metric 变量，检测每一轮模型指标是否提升，如果提升则将更新的参数保存在本地。代码如下：

```

1  best_val_loss = float('inf')
2  epochs = 40
3  best_val_recall = 0
4  val_loss = 0
5  train_loss_history = []
6  val_loss_history = []
7
8  # 定义检查点路径和恢复训练标志
9  checkpoint_path = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
    best_metric_model.pth"
10 resume_training = False # 是否从检查点恢复训练
11
12 # 初始化训练相关变量（会被检查点覆盖）
13 start_epoch = 0
14 best_val_recall = 0
15 best_val_loss = float('inf')
16 train_loss_history = []
17 val_loss_history = []
18
19 # 加载检查点
20 if os.path.exists(checkpoint_path):
21     print(f"发现检查点文件 {checkpoint_path}, 正在加载...")
22     checkpoint = torch.load(checkpoint_path, map_location=device, weights_only= False)
23
24     # 加载模型和优化器状态
25     img_encoder.load_state_dict(checkpoint['img_encoder_state_dict'])
26     txt_encoder.load_state_dict(checkpoint['txt_encoder_state_dict'])
27     optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
28
29     # 加载训练状态
30     start_epoch = checkpoint['epoch']
31     best_val_recall = checkpoint['best_val_recall_at_1']
32     best_val_loss = checkpoint['best_val_loss']
33     train_loss_history = checkpoint.get('train_loss_history', [])
34     val_loss_history = checkpoint.get('val_loss_history', [])
35
36     # 加载tokenizer词表（如果需要）
37     tokenizer.word2idx = checkpoint['tokenizer_vocab']
38
39     resume_training = True
40     print(f"成功加载检查点, 将从 epoch {start_epoch} 继续训练")
41     print(f"当前最佳 Recall@1: {best_val_recall*100:.2f}% | 最佳验证损失: {best_val_loss:.4f}")
42
43 for epoch in range(epochs):
44     img_encoder.train() # 设置为训练模式
45     txt_encoder.train() # 设置为训练模式
46     epoch_loss = 0.0
47     pbar = tqdm(train_dataloader, desc=f"Epoch {epoch+1}/{epochs}", unit = "batch")
48     for images, captions_ids in pbar:
49         images = images.to(device)
50         captions_ids = captions_ids.to(device)
51
52         image_embeds = img_encoder(images) # [batch, embed_dim]
53         text_embeds = txt_encoder(captions_ids) # [batch, embed_dim]
54         loss = contrastive_loss(image_embeds, text_embeds)
55
56         optimizer.zero_grad()
57         loss.backward()
58         optimizer.step()
59

```

```

60         epoch_loss += loss.item()
61         pbar.set_postfix(loss = loss.item())
62
63         avg_train_loss = epoch_loss / len(train_dataloader)
64         train_loss_history.append(avg_train_loss) # 记录训练损失
65         val_loss = calc_val_loss(img_encoder,txt_encoder,val_dataloader,device)
66         val_loss_history.append(val_loss) # 记录验证损失
67
68         r_txt2img, r_img2txt = evaluate_top_k(img_encoder, txt_encoder, val_dataloader, device
        , topk=(1, 5, 10))
69
70         print(f" Epoch [{epoch+1}/{epochs}]: Train Loss: {avg_train_loss:.4f} | Val Loss: {
        val_loss:.4f}")
71         print(f" 验证集 文本->图像 Recall: @1 {r_txt2img[0]*100:.2f}% | @5 {r_txt2img
        [1]*100:.2f}% | @10 {r_txt2img[2]*100:.2f}%")
72         print(f" 验证集 图像->文本 Recall: @1 {r_img2txt[0]*100:.2f}% | @5 {r_img2txt
        [1]*100:.2f}% | @10 {r_img2txt[2]*100:.2f}%")
73
74         # 如果验证集有改善, 则保存最佳模型, 这里选择loss以及recall参数
75         improved_metric = False
76         if r_txt2img[0] >= best_val_recall:
77             best_val_recall = r_txt2img[0]
78             print(f" > 验证集 Recall@1 提升至: {best_val_recall*100:.2f}%")
79             improved_metric = True
80
81         if val_loss <= best_val_loss:
82             best_val_loss = val_loss
83             print(f" > 验证集损失降低至: {best_val_loss:.4f}")
84             improved_metric = True
85
86         if improved_metric:
87             checkpoint = {
88                 'epoch': epoch + 1,
89                 'img_encoder_state_dict': img_encoder.state_dict(),
90                 'txt_encoder_state_dict': txt_encoder.state_dict(),
91                 'optimizer_state_dict': optimizer.state_dict(),
92                 'tokenizer_vocab': tokenizer.word2idx, # 保存词表
93                 'best_val_recall_at_1': best_val_recall,
94                 'best_val_loss': best_val_loss,
95                 'embed_dim': embed_dim, # 保存模型参数
96                 'vocab_size': vocab_size # 保存模型参数
97             }
98             torch.save(checkpoint, checkpoint_path)
99             print(f" > Best model updated at epoch {epoch+1} ")
100
101         print("\n训练完成。")
102
103         #绘制loss曲线
104         plt.figure(figsize=(10, 6))
105         plt.plot(range(1, epochs + 1), train_loss_history, label='平均训练损失 (Avg Train Loss)')
106         plt.plot(range(1, epochs + 1), val_loss_history, label='平均验证损失 (Avg Val Loss)')
107         plt.xlabel('轮次 (Epochs)')
108         plt.ylabel('损失 (Loss)')
109         plt.title('训练和验证损失随轮次变化曲线')
110         plt.legend() # 显示图例
111         plt.grid(True) # 显示网格
112         plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/loss_curve.
        png") # 将图像保存到文件
113         print("损失曲线已保存为 loss_curve.png")
114         plt.show()

```

3.5 Task4: 结果分析与可视化

模型经过训练后,需要在测试集上检测其性能。设置 epoch=40, 每一个 epoch 中的 batch size 设置为 4, 最终得到的模型性能指标如下:

```
> 验证集 Recall@5 提升至: 11.30%
> Best model updated at epoch 39
Epoch 40/40: 100%
Extracting embeddings: 100%
1811/1812 [00:25<00:00, 1.70batch/s, loss=0.134] | 1812/1812 [00:45<00:00, 22.02it/s]

Test - Image Retrieval:
Recall@1: 18.81%
Recall@5: 28.73%
Recall@10: 40.30%
Epoch [40/40]: Train loss: 0.0815 | Val loss: 0.2801
验证集 文本->图像 Recall: @1 18.81% | @5 28.73% | @10 40.30%
验证集 图像->文本 Recall: @1 11.32% | @5 29.65% | @10 40.27%

训练完成
损失曲线已保存为 loss_curve.png
```

图 2: 训练 40 个周期后模型的性能指标

而绘制出的 loss 曲线如下图:

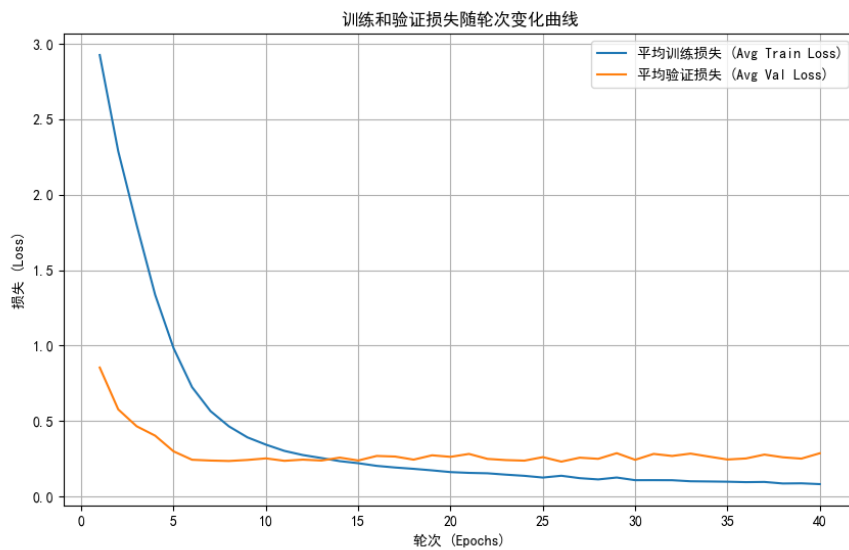


图 3: 训练 40 个周期的 loss 曲线

为了可视化文本 → 图像 Top-K 检索示例, 我新建了一个 test.py 文件, 在原本 evaluate_top_k 函数的基础上编写了 visualize_top_k 函数, 其主要实现逻辑为: 设置一个随机数, 将该随机数对应的 test_caption.txt 文件中提供的描述提取出来作为 Query, 并将相似度排名前五的图片展示出来, 代码如下:

```
1 def visualize_top_k(query_idx, img_encoder, txt_encoder, dataset, device, k=5):
2     img_encoder.eval() # 设置为评估模式
3     txt_encoder.eval() # 设置为评估模式
4
5     # 获取查询样本
6     query_image, query_caption_ids = dataset[query_idx]
7     query_caption = dataset.pairs[query_idx][1]
8
9     # 提取所有图像嵌入
10    all_image_embeds, all_img_paths = [], []
11    with torch.no_grad():
12        for img_path in dataset.img_paths:
13            image = Image.open(os.path.join(dataset.root_dir, img_path)).convert("RGB")
14            image = dataset.transform(image).unsqueeze(0).to(device)
15            embed = img_encoder(image).cpu()
16            all_image_embeds.append(embed)
```

```

17     all_img_paths.append(img_path)
18     all_image_embeds = torch.cat(all_image_embeds)
19
20     # 提取查询文本嵌入
21     with torch.no_grad():
22         caption_ids = query_caption_ids.unsqueeze(0).to(device)
23         text_embed = txt_encoder(caption_ids).cpu()
24
25     # 计算相似度
26     similarities = F.cosine_similarity(text_embed, all_image_embeds, dim=1)
27     topk_indices = similarities.argsort(descending=True)[:k]
28
29     # 可视化
30     plt.figure(figsize=(15, 5))
31     plt.subplot(1, k + 1, 1)
32     plt.imshow(Image.open(os.path.join(dataset.root_dir, dataset.img_paths[query_idx])))
33     plt.title("Query Image\n" + query_caption)
34     plt.axis('off')
35
36     for i, idx in enumerate(topk_indices):
37         plt.subplot(1, k+1, i+2)
38         img = Image.open(os.path.join(dataset.root_dir, all_img_paths[idx]))
39         plt.imshow(img)
40         plt.title(f"Rank {i+1}\nSimilarity: {similarities[idx]:.2f}")
41         plt.axis('off')
42     plt.tight_layout()
43     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
44     retrieval_example.png")
45     plt.show()

```

得到的结果如下图所示：



图 4: Top-5 相似图像结果

在测试集上的性能表现如下：

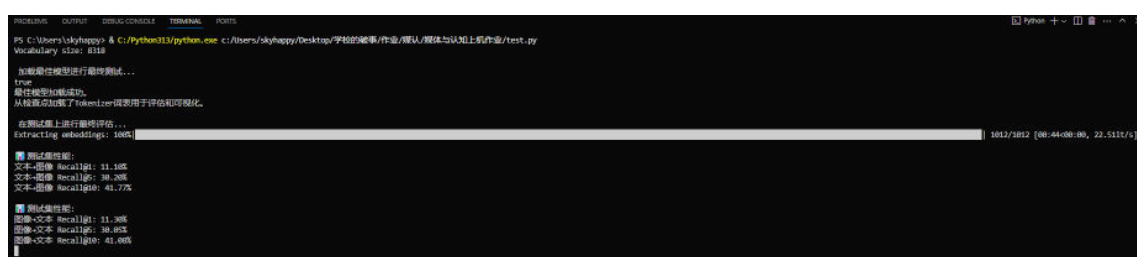


图 5: 在测试集上的模型性能

可以看到，提出的 Query 是一个和成年人一块走的小男孩，而排名前五的图片中均有小男孩出现，特别是第一和第三张图片，是符合 Query 要求的。剩余的三张图片只有一个小男孩，说明在检索时 query 的“little boy”肯定得到了极高的注意力得分，这可能与选择的模型以及训练过程中文本描述的多样性比较有限有关。同时，训练数据可能存在偏差，如一些场景或文本描述占比失衡，导致模型对少见情况泛化能力差。

为了提高模型表现性能，可以尝试更先进的模型结构或改进现有结构，如改进特征提取器，采用更复杂的注意力机制加强文本 - 图像特征交互；或者利用大规模公开数据集进行预训练，再在 Flickr8k 数据集上微调，提升模型泛化能力和对特定任务的适应性；还可以选用其他更为合适的对比学习损失函数，比如 Triplet loss 等等；最后还可以通过随即搜索算法、贝叶斯优化等方式对训练时超参数的设计进行优化，提高模型训练效果。

为了实现 T-SNE 嵌入空间的可视化，我在 test.py 中编写了 tsne_visualize 函数，代码如下：

```

1 def tsne_visualize(img_encoder, txt_encoder, dataloader, device, n_samples=200):
2     """TSNE嵌入空间可视化"""
3     img_encoder.eval() # 设置为评估模式
4     txt_encoder.eval() # 设置为评估模式
5
6     # 收集嵌入
7     image_embeds, text_embeds, labels = [], [], []
8     with torch.no_grad():
9         for idx, (images, captions_ids) in enumerate(dataloader):
10             if idx * dataloader.batch_size >= n_samples:
11                 break
12             images = images.to(device)
13             captions_ids = captions_ids.to(device)
14
15             image_embeds.append(img_encoder(images).cpu())
16             text_embeds.append(txt_encoder(captions_ids).cpu())
17             labels.extend([f"Sample {idx*dataloader.batch_size+i}" for i in range(len(images))
18                             ])
19
20     # 合并并降维
21     combined_embeds = torch.cat(image_embeds + text_embeds).numpy()
22     tsne = TSNE(n_components=2, random_state=42)
23     reduced = tsne.fit_transform(combined_embeds)
24
25     # 绘图
26     plt.figure(figsize=(12, 8))
27     split_point = len(image_embeds)
28     plt.scatter(reduced[:split_point, 0], reduced[:split_point, 1], c='r', label='Images')
29     plt.scatter(reduced[split_point:, 0], reduced[split_point:, 1], c='b', label='Texts')
30     plt.title("T-SNE Visualization of Embedding Space")
31     plt.xlabel("TSNE-1")
32     plt.ylabel("TSNE-2")
33     plt.legend()
34     plt.grid(alpha=0.3)
35     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/tsne_visualization.png")
36     plt.show()

```

运行后结果如图所示：

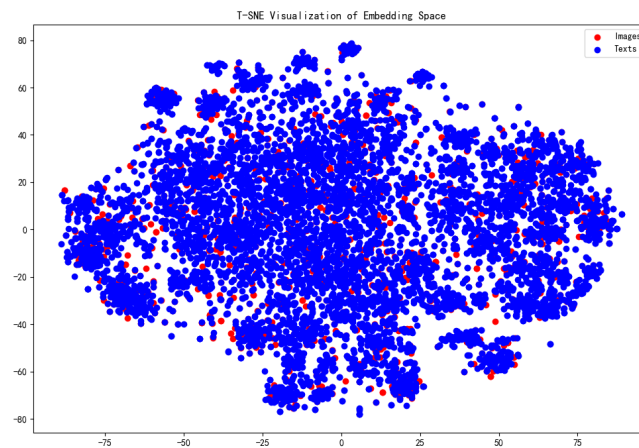


图 6: T-SNE 可视化结果

4 实验总结

中期验收前，主要将模型的基础模块代码完善并编写好了训练和测试代码，对 ResNet、LSTM、Transformer 的底层原理更加了解，同时自行编写 loss、train 和 test 代码也让我对训练模型的过程更加了解。在最终验收前，还需要做一些改进优化的工作，如模型结构优化、对数据增强与正则化、调节损失函数与学习率等，使得最终性能得到提高。

最后感谢老师、助教在实验中的细心指导与帮助！

5 源代码

```
1 # train_and_eval.py
2
3 import os
4 from torch.utils.data import DataLoader
5 from models.image_encoder import ImageEncoder
6 from models.text_encoder import TextEncoder
7 from loss import contrastive_loss
8 from data_loader import Flickr8kDataset
9 from utils import SimpleTokenizer
10 import torch
11 import torch.nn.functional as F
12 from tqdm import tqdm
13 import numpy as np
14 import matplotlib.pyplot as plt
15 from PIL import Image
16 from sklearn.manifold import TSNE
17 import random
18
19 # 供同学们参考
20
21 plt.rcParams['font.sans-serif'] = ['SimHei']
22 plt.rcParams['axes.unicode_minus'] = False
23
24 def evaluate_top_k(img_encoder, txt_encoder, dataloader, device, topk=(1, 5, 10)):
25     img_encoder.eval()
26     txt_encoder.eval()
27
28     all_image_embeds = []
29     all_text_embeds = []
30
31     with torch.no_grad():
32         for images, captions_ids in tqdm(dataloader, desc="Extracting embeddings"):
33             images = images.to(device)
34             captions_ids = captions_ids.to(device)
35
36             image_embed = img_encoder(images) # [1, dim]
37             text_embed = txt_encoder(captions_ids) # [1, dim]
38
39             all_image_embeds.append(image_embed.cpu())
40             all_text_embeds.append(text_embed.cpu())
41
42     all_image_embeds = torch.cat(all_image_embeds, dim=0) # [N, D]
43     all_text_embeds = torch.cat(all_text_embeds, dim=0) # [N, D]
44
45     # 归一化
46     all_image_embeds = F.normalize(all_image_embeds, dim=1)
47     all_text_embeds = F.normalize(all_text_embeds, dim=1)
48
49     # 文本 -> 图像检索
50     sim_matrix = torch.matmul(all_text_embeds, all_image_embeds.T) # [N, N]
51     txt2img_ranks = torch.argsort(sim_matrix, dim=1, descending=True)
52
53     # 图像 -> 文本检索
54     sim_matrix_T = sim_matrix.T # [N, N]
55     img2txt_ranks = torch.argsort(sim_matrix_T, dim=1, descending=True)
56
57     def recall_at_k(ranks, topk):
58         recalls = []
59         for k in topk:
```

```

60         match = [i in ranks[i][:k] for i in range(len(ranks))]
61         recalls.append(np.mean(match))
62         return recalls
63
64     r_txt2img = recall_at_k(txt2img_ranks, topk)
65     r_img2txt = recall_at_k(img2txt_ranks, topk)
66
67     print("\n Text → Image Retrieval:")
68     for i, k in enumerate(topk):
69         print(f"Recall@{k}: {r_txt2img[i]*100:.2f}%")
70
71     print("\n Image → Text Retrieval:")
72     for i, k in enumerate(topk):
73         print(f"Recall@{k}: {r_img2txt[i]*100:.2f}%")
74
75     return r_txt2img, r_img2txt
76
77 def calc_val_loss(img_encoder, txt_encoder, dataloader, device):
78     img_encoder.eval() # 设置为评估模式
79     txt_encoder.eval() # 设置为评估模式
80     total_val_loss = 0 # 使用 total_val_loss 累加
81     with torch.no_grad(): # 在评估阶段不计算梯度
82         pbar = tqdm(dataloader, desc="计算验证损失", unit="batch", leave=False)
83         for images, captions_ids in pbar:
84             images = images.to(device)
85             captions_ids = captions_ids.to(device)
86
87             image_embeds = img_encoder(images)
88             text_embeds = txt_encoder(captions_ids)
89             loss = contrastive_loss(image_embeds, text_embeds) # 调用对比损失函数
90             total_val_loss += loss.item()
91             pbar.set_postfix(loss=loss.item()) # 显示当前batch的损失
92     avg_val_loss = total_val_loss / len(dataloader)
93     return avg_val_loss
94
95
96 def main():
97     # 设备设置
98     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
99
100     # 文件路径, 根据实际调整
101     token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
102     captions.txt" # 总的 captions 文件, 用于构建词表
103     train_token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
104     Flickr8k/train_captions.txt" # 训练集, 格式: image,caption
105     val_token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
106     Flickr8k/val_captions.txt" # 验证集
107
108     # 读取所有 caption 用于构建总词表 (假设以 tab 分隔, 如果不是, 请修改 split 参数)
109     with open(token_file, 'r', encoding="utf-8") as f:
110         lines = f.readlines()
111     captions = [line.strip().split(',')[1] for line in lines if line.strip()]
112
113     # 构建统一的 tokenizer
114     tokenizer = SimpleTokenizer(captions, min_freq=1)
115     vocab_size = len(tokenizer)
116     print(f"Vocabulary size: {vocab_size}")
117
118     # 构建数据集与 DataLoader: 训练集、验证集、测试集
119     train_dataset = Flickr8kDataset(
120         root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
121         images", # 图片所在目录

```

```

118     captions_file=train_token_file,    # 训练集 captions 文件, 格式: image<TAB>caption
119     tokenizer=tokenizer
120 )
121 val_dataset = Flickr8kDataset(
122     root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
123     images",
124     captions_file=val_token_file,      # 验证集 captions 文件
125     tokenizer=tokenizer
126 )
127
128 train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=4,
129                                drop_last=True)
130
131 # 为保证评估稳定, 每个 batch 使用 batch_size=1
132 val_dataloader = DataLoader(val_dataset, batch_size=4, shuffle=False, num_workers=4,
133                                drop_last=False)
134
135 # 构造模型 (设定 embed_dim=256)
136 embed_dim = 256
137 img_encoder = ImageEncoder(embed_dim=embed_dim).to(device)
138 txt_encoder = TextEncoder(vocab_size, embed_dim=embed_dim).to(device)
139
140 # 优化器
141 optimizer = torch.optim.Adam(
142     list(img_encoder.parameters()) + list(txt_encoder.parameters()),
143     lr=1e-4
144 )
145
146 best_val_loss = float('inf')
147 epochs = 40
148 best_val_recall = 0
149 val_loss = 0
150 train_loss_history = []
151 val_loss_history = []
152
153 # 新增代码: 定义检查点路径和恢复训练标志
154 checkpoint_path = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
155 best_metric_model.pth"
156 resume_training = False # 是否从检查点恢复训练
157
158 # 新增代码: 初始化训练相关变量 (会被检查点覆盖)
159 start_epoch = 0
160 best_val_recall = 0
161 best_val_loss = float('inf')
162 train_loss_history = []
163 val_loss_history = []
164
165 # 加载检查点 -----
166 if os.path.exists(checkpoint_path):
167     print(f"发现检查点文件 {checkpoint_path}, 正在加载...")
168     checkpoint = torch.load(checkpoint_path, map_location=device, weights_only= False)
169
170     # 加载模型和优化器状态
171     img_encoder.load_state_dict(checkpoint['img_encoder_state_dict'])
172     txt_encoder.load_state_dict(checkpoint['txt_encoder_state_dict'])
173     optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
174
175     # 加载训练状态
176     start_epoch = checkpoint['epoch']
177     best_val_recall = checkpoint['best_val_recall_at_1']
178     best_val_loss = checkpoint['best_val_loss']
179     train_loss_history = checkpoint.get('train_loss_history', [])
180     val_loss_history = checkpoint.get('val_loss_history', [])

```

```

176
177     # 加载tokenizer词表（如果需要）
178     tokenizer.word2idx = checkpoint['tokenizer_vocab']
179
180     resume_training = True
181     print(f"成功加载检查点，将从 epoch {start_epoch} 继续训练")
182     print(f"当前最佳 Recall@1: {best_val_recall*100:.2f}% | 最佳验证损失: {best_val_loss:.4f}")
183
184     for epoch in range(epochs):
185         img_encoder.train()      # 设置为训练模式
186         txt_encoder.train()      # 设置为训练模式
187         epoch_loss = 0.0
188         pbar = tqdm(train_dataloader, desc=f"Epoch {epoch+1}/{epochs}", unit = "batch")
189         for images, captions_ids in pbar:
190             images = images.to(device)
191             captions_ids = captions_ids.to(device)
192
193             image_embeds = img_encoder(images)      # [batch, embed_dim]
194             text_embeds = txt_encoder(captions_ids) # [batch, embed_dim]
195             loss = contrastive_loss(image_embeds, text_embeds)
196
197             optimizer.zero_grad()
198             loss.backward()
199             optimizer.step()
200
201             epoch_loss += loss.item()
202             pbar.set_postfix(loss = loss.item())
203
204         avg_train_loss = epoch_loss / len(train_dataloader)
205         train_loss_history.append(avg_train_loss) # 记录训练损失
206         val_loss = calc_val_loss(img_encoder, txt_encoder, val_dataloader, device)
207         val_loss_history.append(val_loss) # 记录验证损失
208
209         r_txt2img, r_img2txt = evaluate_top_k(img_encoder, txt_encoder, val_dataloader, device
210 , topk=(1, 5, 10))
211
212         print(f" Epoch [{epoch+1}/{epochs}]: Train Loss: {avg_train_loss:.4f} | Val Loss: {val_loss:.4f}")
213         print(f" 验证集 文本->图像 Recall: @1 {r_txt2img[0]*100:.2f}% | @5 {r_txt2img[1]*100:.2f}% | @10 {r_txt2img[2]*100:.2f}%")
214         print(f" 验证集 图像->文本 Recall: @1 {r_img2txt[0]*100:.2f}% | @5 {r_img2txt[1]*100:.2f}% | @10 {r_img2txt[2]*100:.2f}%")
215
216         # 如果验证集有改善，则保存最佳模型，这里选择loss以及recall参数
217         improved_metric = False
218         if r_txt2img[0] >= best_val_recall:
219             best_val_recall = r_txt2img[0]
220             print(f" > 验证集 Recall@1 提升至: {best_val_recall*100:.2f}%")
221             improved_metric = True
222
223         if val_loss <= best_val_loss:
224             best_val_loss = val_loss
225             print(f" > 验证集损失降低至: {best_val_loss:.4f}")
226             improved_metric = True
227
228         if improved_metric:
229             checkpoint = {
230                 'epoch': epoch + 1,
231                 'img_encoder_state_dict': img_encoder.state_dict(),
232                 'txt_encoder_state_dict': txt_encoder.state_dict(),
233                 'optimizer_state_dict': optimizer.state_dict(),

```

```

233         'tokenizer_vocab': tokenizer.word2idx, # 保存词表
234         'best_val_recall_at_1': best_val_recall,
235         'best_val_loss': best_val_loss,
236         'embed_dim': embed_dim, # 保存模型参数
237         'vocab_size': vocab_size # 保存模型参数
238     }
239     torch.save(checkpoint, checkpoint_path)
240     print(f"    > Best model updated at epoch {epoch+1} ")
241
242     print("\n训练完成。")
243
244     #绘制loss曲线
245     plt.figure(figsize=(10, 6))
246     plt.plot(range(1, epochs + 1), train_loss_history, label='平均训练损失 (Avg Train Loss)')
247     plt.plot(range(1, epochs + 1), val_loss_history, label='平均验证损失 (Avg Val Loss)')
248     plt.xlabel('轮次 (Epochs)')
249     plt.ylabel('损失 (Loss)')
250     plt.title('训练和验证损失随轮次变化曲线')
251     plt.legend() # 显示图例
252     plt.grid(True) # 显示网格
253     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/loss_curve.
254     png") # 将图像保存到文件
255     print("损失曲线已保存为 loss_curve.png")
256     plt.show()
257
258 if __name__ == "__main__":
259     main()

```

Listing 1: train.py

```

1  # test_and_visualization.py
2
3  import os
4  from torch.utils.data import DataLoader
5  from models.image_encoder import ImageEncoder
6  from models.text_encoder import TextEncoder
7  from loss import contrastive_loss
8  from data_loader import Flickr8kDataset
9  from utils import SimpleTokenizer
10 import torch
11 import torch.nn.functional as F
12 from tqdm import tqdm
13 import numpy as np
14 import matplotlib.pyplot as plt
15 from PIL import Image
16 from sklearn.manifold import TSNE
17 import random
18
19 #供同学们参考
20
21 plt.rcParams['font.sans-serif'] = ['SimHei']
22 plt.rcParams['axes.unicode_minus'] = False
23
24 def evaluate_top_k(img_encoder, txt_encoder, dataloader, device, topk=(1, 5, 10)):
25     img_encoder.eval()
26     txt_encoder.eval()
27
28     all_image_embeds = []
29     all_text_embeds = []
30
31     with torch.no_grad():

```

```

32     for images, captions_ids in tqdm(dataloader, desc="Extracting embeddings"):
33         images = images.to(device)
34         captions_ids = captions_ids.to(device)
35
36         image_embed = img_encoder(images) # [1, dim]
37         text_embed = txt_encoder(captions_ids) # [1, dim]
38
39         all_image_embeds.append(image_embed.cpu())
40         all_text_embeds.append(text_embed.cpu())
41
42     all_image_embeds = torch.cat(all_image_embeds, dim=0) # [N, D]
43     all_text_embeds = torch.cat(all_text_embeds, dim=0) # [N, D]
44
45     # 归一化
46     all_image_embeds = F.normalize(all_image_embeds, dim=1)
47     all_text_embeds = F.normalize(all_text_embeds, dim=1)
48
49     # 文本 -> 图像检索
50     sim_matrix = torch.matmul(all_text_embeds, all_image_embeds.T) # [N, N]
51     txt2img_ranks = torch.argsort(sim_matrix, dim=1, descending=True)
52
53     # 图像 -> 文本检索
54     sim_matrix_T = sim_matrix.T # [N, N]
55     img2txt_ranks = torch.argsort(sim_matrix_T, dim=1, descending=True)
56
57     def recall_at_k(ranks, topk):
58         recalls = []
59         for k in topk:
60             match = [i in ranks[i][:k] for i in range(len(ranks))]
61             recalls.append(np.mean(match))
62         return recalls
63
64     r_txt2img = recall_at_k(txt2img_ranks, topk)
65     r_img2txt = recall_at_k(img2txt_ranks, topk)
66
67     print("\n Text -> Image Retrieval:")
68     for i, k in enumerate(topk):
69         print(f"Recall@{k}: {r_txt2img[i]*100:.2f}%")
70
71
72     return r_txt2img, r_img2txt
73
74 def visualize_top_k(query_idx, img_encoder, txt_encoder, dataset, device, k=5):
75     img_encoder.eval() # 设置为评估模式
76     txt_encoder.eval() # 设置为评估模式
77
78     # 获取查询样本
79     query_image, query_caption_ids = dataset[query_idx]
80     query_caption = dataset.pairs[query_idx][1]
81
82     # 提取所有图像嵌入
83     all_image_embeds, all_img_paths = [], []
84     with torch.no_grad():
85         for img_path in dataset.img_paths:
86             image = Image.open(os.path.join(dataset.root_dir, img_path)).convert("RGB")
87             image = dataset.transform(image).unsqueeze(0).to(device)
88             embed = img_encoder(image).cpu()
89             all_image_embeds.append(embed)
90             all_img_paths.append(img_path)
91     all_image_embeds = torch.cat(all_image_embeds)
92
93     # 提取查询文本嵌入

```

```

94     with torch.no_grad():
95         caption_ids = query_caption_ids.unsqueeze(0).to(device)
96         text_embed = txt_encoder(caption_ids).cpu()
97
98     # 计算相似度
99     similarities = F.cosine_similarity(text_embed, all_image_embeds, dim=1)
100     topk_indices = similarities.argsort(descending=True)[:k]
101
102     # 可视化
103     plt.figure(figsize=(15, 5))
104     plt.subplot(1, k + 1, 1)
105     plt.imshow(Image.open(os.path.join(dataset.root_dir, dataset.img_paths[query_idx])))
106     plt.title("Query Image\n" + query_caption)
107     plt.axis('off')
108
109     for i, idx in enumerate(topk_indices):
110         plt.subplot(1, k+1, i+2)
111         img = Image.open(os.path.join(dataset.root_dir, all_img_paths[idx]))
112         plt.imshow(img)
113         plt.title(f"Rank {i+1}\nSimilarity: {similarities[idx]:.2f}")
114         plt.axis('off')
115     plt.tight_layout()
116     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
117               retrieval_example.png")
118     plt.show()
119
120 def tsne_visualize(img_encoder, txt_encoder, dataloader, device, n_samples=200):
121     """TSNE嵌入空间可视化"""
122     img_encoder.eval() # 设置为评估模式
123     txt_encoder.eval() # 设置为评估模式
124
125     # 收集嵌入
126     image_embeds, text_embeds, labels = [], [], []
127     with torch.no_grad():
128         for idx, (images, captions_ids) in enumerate(dataloader):
129             if idx * dataloader.batch_size >= n_samples:
130                 break
131             images = images.to(device)
132             captions_ids = captions_ids.to(device)
133
134             image_embeds.append(img_encoder(images).cpu())
135             text_embeds.append(txt_encoder(captions_ids).cpu())
136             labels.extend([f"Sample {idx*dataloader.batch_size+i}" for i in range(len(images))
137                           ])
138
139     # 合并并降维
140     combined_embeds = torch.cat(image_embeds + text_embeds).numpy()
141     tsne = TSNE(n_components=2, random_state=42)
142     reduced = tsne.fit_transform(combined_embeds)
143
144     # 绘图
145     plt.figure(figsize=(12, 8))
146     split_point = len(image_embeds)
147     plt.scatter(reduced[:split_point, 0], reduced[:split_point, 1], c='r', label='Images')
148     plt.scatter(reduced[split_point:, 0], reduced[split_point:, 1], c='b', label='Texts')
149     plt.title("T-SNE Visualization of Embedding Space")
150     plt.xlabel("TSNE-1")
151     plt.ylabel("TSNE-2")
152     plt.legend()
153     plt.grid(alpha=0.3)
154     plt.savefig("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
155               tsne_visualization.png")

```

```

153 plt.show()
154
155
156
157 def calc_val_loss(img_encoder, txt_encoder, dataloader, device):
158     img_encoder.eval() # 设置为评估模式
159     txt_encoder.eval() # 设置为评估模式
160     total_val_loss = 0 # 使用 total_val_loss 累加
161     with torch.no_grad(): # 在评估阶段不计算梯度
162         pbar = tqdm(dataloader, desc="计算验证损失", unit="batch", leave=False)
163         for images, captions_ids in pbar:
164             images = images.to(device)
165             captions_ids = captions_ids.to(device)
166
167             image_embeds = img_encoder(images)
168             text_embeds = txt_encoder(captions_ids)
169             loss = contrastive_loss(image_embeds, text_embeds) # 调用对比损失函数
170             total_val_loss += loss.item()
171             pbar.set_postfix(loss=loss.item()) # 显示当前batch的损失
172     avg_val_loss = total_val_loss / len(dataloader)
173     return avg_val_loss
174
175
176 def main():
177     # 设备设置
178     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
179
180     # 文件路径, 根据实际调整
181     token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
182     captions.txt" # 总的 captions 文件, 用于构建词表
183
184     test_token_file = "C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/
185     Flickr8k/test_captions.txt" # 测试集
186
187     # 读取所有 caption 用于构建总词表 (假设以 tab 分隔, 如果不是, 请修改 split 参数)
188     with open(token_file, 'r', encoding="utf-8") as f:
189         lines = f.readlines()
190     captions = [line.strip().split(',')[1] for line in lines if line.strip()]
191
192     # 构建统一的 tokenizer
193     tokenizer = SimpleTokenizer(captions, min_freq=1)
194     vocab_size = len(tokenizer)
195     print(f"Vocabulary size: {vocab_size}")
196
197     test_dataset = Flickr8kDataset(
198         root_dir="C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业/Flickr8k/
199         images",
200         captions_file=test_token_file, # 测试集 captions 文件
201         tokenizer=tokenizer
202     )
203
204     test_dataloader = DataLoader(test_dataset, batch_size=4, shuffle=False, num_workers=4,
205     drop_last=False)
206
207     # 构造模型 (设定 embed_dim=256)
208     embed_dim = 256
209
210     # 训练完成, 最终在测试集上评估
211     print("\n 加载最佳模型进行最终测试...")
212     checkpoint = torch.load("C:/Users/skyhappy/Desktop/学校的破事/作业/媒认/媒体与认知上机作业
213     /best_clip_model.pth", map_location=device, weights_only = False) #加载最优模型
214
215     # 重新初始化模型结构以加载状态字典

```

```

210 loaded_embed_dim = checkpoint.get('embed_dim', embed_dim)
211 loaded_vocab_size = checkpoint.get('vocab_size', vocab_size)
212
213 final_img_encoder = ImageEncoder(embed_dim=loaded_embed_dim).to(device)
214 final_txt_encoder = TextEncoder(vocab_size=loaded_vocab_size, embed_dim=loaded_embed_dim).
    to(device)
215 final_img_encoder.load_state_dict(checkpoint['img_encoder_state_dict'])
216 final_txt_encoder.load_state_dict(checkpoint['txt_encoder_state_dict'])
217 print("最佳模型加载成功。")
218 tokenizer_for_eval_vis = SimpleTokenizer([], min_freq=1) # 创建空tokenizer
219 tokenizer_for_eval_vis.word2idx = checkpoint['tokenizer_vocab']
220
221 # 如果SimpleTokenizer依赖idx2word, 也需要重建
222 if hasattr(tokenizer_for_eval_vis, 'idx2word'):
223     tokenizer_for_eval_vis.idx2word = {v: k for k, v in tokenizer_for_eval_vis.
        word2idx.items()}
224 print("从检查点加载了Tokenizer词表用于评估和可视化。")
225
226 # 使用加载的最佳模型进行评估
227 img_to_eval = final_img_encoder
228 txt_to_eval = final_txt_encoder
229 print("\n 在测试集上进行最终评估...")
230 r_txt2img_test, r_img2txt_test = evaluate_top_k(img_to_eval, txt_to_eval, test_dataloader,
    device)
231
232 print("\n 测试集性能:")
233 print(f"文本→图像 Recall@1: {r_txt2img_test[0]*100:.2f}%")
234 print(f"文本→图像 Recall@5: {r_txt2img_test[1]*100:.2f}%")
235 print(f"文本→图像 Recall@10: {r_txt2img_test[2]*100:.2f}%")
236
237 print("\n 测试集性能:")
238 print(f"图像→文本 Recall@1: {r_img2txt_test[0]*100:.2f}%")
239 print(f"图像→文本 Recall@5: {r_img2txt_test[1]*100:.2f}%")
240 print(f"图像→文本 Recall@10: {r_img2txt_test[2]*100:.2f}%")
241
242 # 可视化示例
243 random_idx = random.randint(0, len(test_dataset)-1)
244 visualize_top_k(random_idx, img_to_eval, txt_to_eval, test_dataset, device, k=5)
245
246 # T-SNE可视化
247 tsne_visualize(img_to_eval, txt_to_eval, test_dataloader, device)
248
249 if __name__ == "__main__":
250     main()

```

Listing 2: test.py

```

1 import torch
2 import torch.nn.functional as F
3 import tqdm
4 #请在这里写出对比损失函数
5 def contrastive_loss(image_embeds, text_embeds, temperature=0.07):
6     """
7     image_embeds, text_embeds: [batch, embed_dim]
8     """
9     device = image_embeds.device
10    batch_size = image_embeds.shape[0]
11
12    logits = torch.matmul(image_embeds, text_embeds.T) / temperature
13
14    # 创建正样本标签 [0,1,2,...,batch-1]
15    labels = torch.arange(batch_size, device=image_embeds.device)

```

```

16
17 # 计算图像->文本和文本->图像两个方向的交叉熵损失
18 loss_i2t = F.cross_entropy(logits, labels)
19 loss_t2i = F.cross_entropy(logits.T, labels) # 转置矩阵计算反向
20
21 # 最终损失取平均
22 loss = (loss_i2t + loss_t2i) / 2
23
24 return loss

```

Listing 3: loss.py

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 # ----- 任务一: 实现 BasicBlock 模块 -----
5
6 class BasicBlock(nn.Module):
7     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
8         super().__init__()
9         # 实现第一个 3x3 卷积层, 包含 stride、padding=1, bias=False;
10         self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
11                                 padding=1, bias=False)
12
13         # 【填空区域】: 对 conv1 的输出接 BatchNorm
14         self.bn1 = nn.BatchNorm2d(out_channels)
15
16         # 定义 ReLU 激活, inplace 设为 True
17         self.relu = nn.ReLU(inplace=True)
18
19         # 【填空区域】: 实现第二个 3x3 卷积层, 步长默认为1
20         self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1,
21                                 bias=False)
22
23         # conv2 的输出接 BatchNorm
24         self.bn2 = nn.BatchNorm2d(out_channels)
25
26         self.downsample = downsample
27 # 请完成 forward 函数
28 def forward(self, x):
29
30     identity = x
31
32     out = self.conv1(x)
33     out = self.bn1(out)
34     out = self.relu(out)
35
36     out = self.conv2(out)
37     out = self.bn2(out)
38
39     # 如果需要下采样, 对 identity 也进行转换
40     if self.downsample is not None:
41         identity = self.downsample(x)
42
43     out = out + identity # 残差连接
44     out = self.relu(out)
45
46     return out
47
48 class ResNet18(nn.Module):
49     def __init__(self, embed_dim=256):
50         super().__init__()
51         self.in_channels = 64

```

```

49     self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
50     self.bn1 = nn.BatchNorm2d(64)
51     self.relu = nn.ReLU(inplace=True)
52     self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
53     self.layer1 = self._make_layer(64, 2, stride=1)
54     self.layer2 = self._make_layer(128, 2, stride=2)
55     self.layer3 = self._make_layer(256, 2, stride=2)
56     self.layer4 = self._make_layer(512, 2, stride=2)
57     self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
58     self.fc = nn.Linear(512, embed_dim)
59
60     def _make_layer(self, out_channels, blocks, stride):
61         downsample = None
62         if stride != 1 or self.in_channels != out_channels:
63             downsample = nn.Sequential(
64                 nn.Conv2d(self.in_channels, out_channels, kernel_size=1, stride=stride, bias=
False),
65                 nn.BatchNorm2d(out_channels)
66             )
67         layers = [BasicBlock(self.in_channels, out_channels, stride, downsample)]
68         self.in_channels = out_channels
69         for _ in range(1, blocks):
70             layers.append(BasicBlock(out_channels, out_channels))
71         return nn.Sequential(*layers)
72
73     def forward(self, x):
74         x = self.relu(self.bn1(self.conv1(x)))
75         x = self.maxpool(x)
76         x = self.layer1(x)
77         x = self.layer2(x)
78         x = self.layer3(x)
79         x = self.layer4(x)
80         x = self.avgpool(x)
81         x = torch.flatten(x, 1)
82         x = self.fc(x)
83         return F.normalize(x, p=2, dim=1)

```

Listing 4: resnet_custom.py

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 class LSTMTextEncoder(nn.Module):
5     def __init__(self, vocab_size, embed_dim=256, hidden_dim=256, num_layers=1, padding_idx=0)
:
6         super().__init__()
7
8         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
9         self.hidden_dim = hidden_dim
10
11
12         self.W_i = nn.Linear(embed_dim, hidden_dim)
13         # 初始化输入门参数, U_i: 上一个隐状态至隐状态
14         self.U_i = nn.Linear(hidden_dim, hidden_dim)
15
16         # 初始化遗忘门参数, W_f: 输入至隐状态
17         self.W_f = nn.Linear(embed_dim, hidden_dim)
18         # 初始化遗忘门参数, U_f: 上一个隐状态至隐状态
19         self.U_f = nn.Linear(hidden_dim, hidden_dim)
20
21         # 初始化输出门参数, W_o: 输入至隐状态
22         self.W_o = nn.Linear(embed_dim, hidden_dim)

```

```

23     #初始化输出门参数, U_o: 上一个隐状态至隐状态
24     self.U_o = nn.Linear(hidden_dim, hidden_dim)
25
26     # 初始化候选状态参数, W_c: 输入至隐状态
27     self.W_c = nn.Linear(embed_dim, hidden_dim)
28     #初始化候选状态参数, U_c: 上一个隐状态至隐状态
29     self.U_c = nn.Linear(hidden_dim, hidden_dim)
30
31     # 全连接层将最终的隐藏状态映射到 embed_dim (目标嵌入空间)
32     self.fc = nn.Linear(hidden_dim, embed_dim)
33     #请完成forward函数
34     def forward(self, captions):
35         """
36         captions: [B, T], 表示批次 B 中每个句子的 token id 序列, T 为序列长度。
37         """
38         word_embedding = self.embedding(captions) # [B, T, E]
39         B, T, E = word_embedding.shape
40
41         h = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
42         c = torch.zeros(B, self.hidden_dim, device = word_embedding.device)
43
44         for t in range(T):
45             x_t = word_embedding[:, t, :]
46
47             i_t = torch.sigmoid(self.W_i(x_t) + self.U_i(h)) # 输入门
48             f_t = torch.sigmoid(self.W_f(x_t) + self.U_f(h)) # 遗忘门
49             o_t = torch.sigmoid(self.W_o(x_t) + self.U_o(h)) # 输出门
50             c_hat_t = torch.tanh(self.W_c(x_t) + self.U_c(h))
51
52             c = f_t * c + i_t * c_hat_t # 更新细胞状态
53             h = o_t * torch.tanh(c) # 更新隐藏状态
54
55         # 将最终隐状态 h 通过全连接层映射至目标嵌入空间
56         out = self.fc(h) # [B, embed_dim]
57         # 对输出结果进行 L2 正则化归一化, 确保嵌入向量单位化
58         return F.normalize(out, p=2, dim=1)
59

```

Listing 5: lstm_custom.py

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4  import math
5
6  class PositionalEncoding(nn.Module):
7      def __init__(self, d_model, max_len=100):
8          super().__init__()
9          pe = torch.zeros(max_len, d_model)
10         position = torch.arange(0, max_len).unsqueeze(1).float()
11         div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) /
12         d_model))
13         pe[:, 0::2] = torch.sin(position * div_term)
14         pe[:, 1::2] = torch.cos(position * div_term)
15         self.pe = pe.unsqueeze(0)
16
17     def forward(self, x):
18         return x + self.pe[:, :x.size(1)].to(x.device)
19
20     # ----- 填空题 1: 实现 MultiHeadSelfAttention 模块 -----
21     class MultiHeadSelfAttention(nn.Module):

```

```

22 def __init__(self, embed_dim, num_heads):
23     super().__init__()
24     assert embed_dim % num_heads == 0, "embed_dim 必须能整除 num_heads"
25     self.num_heads = num_heads
26     self.head_dim = embed_dim // num_heads
27     self.qkv_proj = nn.Linear(embed_dim, embed_dim * 3)
28     self.out_proj = nn.Linear(embed_dim, embed_dim)
29     # 请完成 forward 函数
30     def forward(self, x, mask=None):
31         # x: [B, T, D], D=embed_dim
32         B, T, D = x.size()
33         qkv = self.qkv_proj(x) # [B, T, 3D]
34         q, k, v = torch.chunk(qkv, 3, dim=-1)
35
36         # 拆分
37         q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2) # [B, nh, T, hd]
38         k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
39         v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
40
41         # 计算注意力分数
42         attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim) # [B, nh, T, T]
43
44         if mask is not None:
45             attn = attn.masked_fill(mask.unsqueeze(1).unsqueeze(1) == 0, float('-inf'))
46         attn = F.softmax(attn, dim=-1)
47
48         # 计算注意力输出
49         attn_output = attn @ v # [B, nh, T, hd]
50         attn_output = attn_output.transpose(1, 2).contiguous().view(B, T, D) # [B, T, D]
51
52         # 通过输出投影层输出结果
53         return self.out_proj(attn_output)
54
55 class TransformerBlock(nn.Module):
56     def __init__(self, embed_dim, num_heads):
57         super().__init__()
58         self.attn = MultiHeadSelfAttention(embed_dim, num_heads)
59         self.ln1 = nn.LayerNorm(embed_dim)
60         # 初始化前馈网络, 先 Linear(embed_dim, embed_dim*4) -> ReLU -> Linear(embed_dim*4,
61         embed_dim)
62         self.ff = nn.Sequential(
63             nn.Linear(embed_dim, embed_dim * 4),
64             nn.ReLU(),
65             nn.Linear(embed_dim * 4, embed_dim)
66         )
67         self.ln2 = nn.LayerNorm(embed_dim)
68
69     def forward(self, x, mask=None):
70         # 残差连接: 先 LayerNorm, 再 MultiHeadSelfAttention, 再加上输入
71         # 计算自注意力块输出, 并加上原输入, 实现残差连接
72         x = x + self.attn(self.ln1(x), mask)
73         # 同理, 对于前馈网络: 先 LayerNorm, 再前馈计算, 再加上输入
74         # 计算前馈网络输出, 并加上残差
75         x = x + self.ff(self.ln2(x))
76         return x
77
78 # ----- 填空题 2: 实现 TransformerTextEncoder 模块 -----
79 class TransformerTextEncoder(nn.Module):
80     def __init__(self, vocab_size, embed_dim=256, num_heads=4, num_layers=4, padding_idx=0,
81         max_len=100):
82         super().__init__()
83         # 初始化嵌入层

```

```

82     self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=padding_idx)
83     # 初始化位置编码器, 参数为 embed_dim 和 max_len
84     self.pos_encoder = PositionalEncoding(embed_dim, max_len=max_len)
85     self.layers = nn.ModuleList([TransformerBlock(embed_dim, num_heads) for _ in range(
num_layers)])
86     self.fc = nn.Linear(embed_dim, embed_dim)
87     # 请完成forward函数
88     def forward(self, captions):
89         # captions: [B, T]
90
91         # 输入嵌入和位置编码
92         x = self.embedding(captions) # [B, T, D]
93         x = self.pos_encoder(x)      # [B, T, D]
94
95         # 通过多层Transformer块
96         for layer in self.layers:
97             x = layer(x)              # [B, T, D]
98
99         # 全局平均池化
100        x = x.mean(dim=1)              # [B, D]
101        out = self.fc(x)
102        return F.normalize(out, p=2, dim=1)
103
104    # ----- 测试单元 -----
105    if __name__ == "__main__":
106        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
107        vocab_size = 1000
108        # 随机生成输入: 形状为 [batch=4, T=20]
109        dummy_captions = torch.randint(0, vocab_size, (4, 20)).to(device)
110        # 实例化 TransformerTextEncoder, 要求使用 4 层 (num_layers=4)
111        model = TransformerTextEncoder(vocab_size, embed_dim=256, num_heads=4, num_layers=4,
padding_idx=0, max_len=100).to(device)
112        output = model(dummy_captions)
113        print("输出嵌入张量的形状: ", output.shape) # 期望输出: [4, 256]

```

Listing 6: transformer_encoder.py